

# Tarea 1: Computación de Alto Rendimiento (IIC3533)

Josefina Abbott Cerda, José Ignacio Salas Coeymans y Borja Smith Vergara

11 de septiembre de 2026

## (a) Generación de datos sintéticos

Para generar los datos sintéticos, seguimos los siguientes pasos:

1. Generar los datos, usando el generador aleatorio de NumPy (`np.random.default_rng`) fijando una semilla base, nosotros la establecimos como 1111.
2. Generar el vector de coeficientes reales  $\beta^* \in \mathbb{R}^{301}$  muestreando de una distribución normal estándar  $\mathcal{N}(0, 1)$ .
3. Crear una matriz base de  $10000 \times 300$  con valores independientes  $\mathcal{N}(0, 1)$ .
4. Concatenar horizontalmente un vector columna de unos a la matriz base para incorporar el intercepto ( $\beta_0$ ), obteniendo así la matriz de diseño  $X \in \mathbb{R}^{10000 \times 301}$ .
5. Calcular el vector de respuesta  $y \in \mathbb{R}^{10000}$  con el producto matricial  $y = X\beta^* + \varepsilon$ , donde  $\varepsilon$  es un vector de ruido con 10000 muestras i.i.d. de  $\mathcal{N}(0, 1)$ .

Para correr el script de generación de datos, se puede ejecutar el siguiente comando en la terminal:

```
python src/data_generation.py
```

## (b) Implementaciones de bootstrapping

### Evolución de `bs_auto.py`

#### 1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Utiliza la configuración por defecto de scikit-learn. Emplea `LinearRegression()` (que calcula un intercepto internamente) y extrae los coeficientes iterando con un ciclo `for` estándar y `.append()`.
- **Versión 2:** Introduce `fit_intercept=False`. Dado que la matriz de diseño  $X$  ya incluye una columna de unos pre-concatenada[cite: 1], esto evita el cálculo de un intercepto redundante.

- **Versión 3:** Reemplaza el ciclo `for` por una comprensión de listas (*list comprehension*) vectorizada para la extracción de coeficientes, mejorando la asignación de memoria.
- **Versión 4:** Añade explícitamente el parámetro `bootstrap=True` al `BaggingRegressor` para garantizar la correctitud del algoritmo (muestreo con reemplazo)[cite: 1], manteniendo la comprensión de listas.
- **Versión 5:** Mantiene los parámetros óptimos del modelo, pero revierte la extracción de coeficientes al ciclo `for` con `.append()` para contrastar el manejo de memoria final.

## 2. Tiempos de ejecución

| Versión | Características Principales                              | Tiempo ( $p = 1$ ) |
|---------|----------------------------------------------------------|--------------------|
| v1      | Intercepto activo, ciclo <code>for</code>                | 4.7474 s           |
| v2      | Sin intercepto, ciclo <code>for</code>                   | 5.1836 s           |
| v3      | Sin intercepto, list comprehension                       | 5.3599 s           |
| v4      | Sin inter., list comp., bootstrap explícito              | 4.2048 s           |
| v5      | Sin inter., ciclo <code>for</code> , bootstrap explícito | 4.1570 s           |

Cuadro 1: Tiempos de ejecución empíricos para distintas iteraciones de `bs_auto.py`.

## 3. Análisis de la versión óptima

La **Versión 4** es computacional y estructuralmente la más óptima.

En primer lugar, al definir `fit_intercept=False`, se elimina una operación matricial redundante que `scikit-learn` realizaría en cada uno de los  $B = 48$  remuestreos, ya que la matriz  $X$  construida ya contempla el intercepto en su primera columna[cite: 1]. Además, declarar `bootstrap=True` asegura el rigor matemático del remuestreo exigido[cite: 1], protegiendo el código contra posibles cambios en los valores por defecto de la librería.

Si bien empíricamente la Versión 5 registró un tiempo marginalmente inferior en esta ejecución específica (4.15s vs 4.20s), esta diferencia de  $\sim 0.05$  segundos es atribuible a la varianza natural del *scheduling* de la CPU en el sistema operativo. A nivel de arquitectura de software, la Versión 4 es superior porque utiliza una comprensión de listas (`[est.coef_ for est in ...]`) en lugar de un `.append()` dinámico. Esto permite que el motor en C de Python asigne el bloque de memoria necesario de una sola vez, en lugar de redimensionar la lista continuamente en cada iteración, previniendo cuellos de botella en la memoria RAM a medida que el proceso escala.

## Evolución de `bs_sklern.py`

### 1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Utiliza el generador aleatorio global (`np.random.choice`) dentro de cada proceso trabajador. Esto provoca que procesos iniciados simultáneamente compartan la misma semilla y extraigan las mismas muestras, arruinando la independencia del bootstrapping. Además, mantiene activado el cálculo del intercepto por defecto.

- **Versión 2:** Aísla la generación de números aleatorios asignando una semilla determinista única (`seed_b`) a cada proceso mediante `np.random.default_rng(seed_b)`. Esto incrementa levemente el tiempo de ejecución por el costo de instanciar 48 generadores distintos, pero es matemáticamente obligatorio para garantizar la validez estadística.
- **Versión 3:** Introduce `fit_intercept=False`. Al igual que en `bs_auto.py`, esto elimina el procesamiento redundante de la matriz, reduciendo los tiempos de ejecución notablemente. Adicionalmente, el retorno estricto de `model.coef_` en lugar del objeto modelo completo evita cuellos de botella por serialización (*pickling*) durante la comunicación entre procesos de `joblib`.

## 2. Tiempos de ejecución

| Versión | Características Principales                     | Tiempo ( $p = 1$ ) |
|---------|-------------------------------------------------|--------------------|
| v1      | RNG global (incorrecto), intercepto activo      | 5.9958 s           |
| v2      | RNG independiente (correcto), intercepto activo | 7.7394 s           |
| v3      | RNG independiente, sin intercepto               | 5.5406 s           |

Cuadro 2: Tiempos de ejecución empíricos para distintas iteraciones de `bs_sklearn.py`.

## 3. Análisis de la versión óptima

La **Versión 3** representa la implementación correcta y más eficiente utilizando scikit-learn de forma explícita. El manejo independiente de las semillas elimina las condiciones de carrera paralela, mientras que la desactivación del intercepto evita cálculos innecesarios sobre la matriz  $X$ , devolviendo el tiempo de ejecución a niveles óptimos ( $\sim 5.54$ s) sin comprometer la correctitud del algoritmo iterativo.

## Evolución de `bs_numpy.py`

### 1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Traduce literalmente la fórmula de Mínimos Cuadrados Ordinarios entregada en el enunciado:  $\hat{\beta} = (X^T X)^{-1} X^T y$  [cite: 1]. Utiliza `np.linalg.inv()` para calcular explícitamente la matriz inversa antes de realizar las multiplicaciones matriciales.
- **Versión 2:** Reemplaza la inversión explícita reescribiendo el problema como la resolución de un sistema de ecuaciones lineales:  $(X^T X)\hat{\beta} = X^T y$  [cite: 1]. Utiliza `np.linalg.solve()` para encontrar los coeficientes de manera directa.

### 2. Tiempos de ejecución

| Versión | Características Principales                              | Tiempo ( $p = 1$ ) |
|---------|----------------------------------------------------------|--------------------|
| v1      | Cálculo explícito con <code>np.linalg.inv()</code>       | 1.0937 s           |
| v2      | Resolución de sistema con <code>np.linalg.solve()</code> | 1.0204 s           |

Cuadro 3: Tiempos de ejecución empíricos para distintas iteraciones de `bs_numpy.py`.

### 3. Análisis de la versión óptima

La **Versión 2** es la implementación óptima y definitiva para esta sección.

Aunque la diferencia de tiempo empírica en una sola ejecución con  $p = 1$  parece marginal ( $\sim 0.07$  segundos), el cambio posee un peso teórico y arquitectónico fundamental. Calcular explícitamente la inversa de una matriz mediante `np.linalg.inv()` es computacionalmente ineficiente y altamente susceptible a errores de redondeo de punto flotante a medida que la dimensionalidad aumenta.

Al utilizar `np.linalg.solve()`, NumPy delega la operación a subrutinas optimizadas de álgebra lineal (LAPACK), las cuales resuelven el sistema de ecuaciones  $(X^T X)\hat{\beta} = X^T y$  [cite: 1] mediante métodos de factorización directa (como descomposición LU o Cholesky) sin llegar a instanciar la matriz inversa en la memoria. Esto garantiza una mayor estabilidad numérica y previene cuellos de botella algorítmicos, preparándonos correctamente para los experimentos de escalabilidad con  $p_{max}$  procesos [cite: 1].

Para correr las implementaciones, se pueden ejecutar los siguientes comandos en la terminal:

```
python src/bs_auto.py
python src/bs_sklearn.py
python src/bs_numpy.py
```

## (c) Correctitud y reproducibilidad

**Consistencia con  $\beta^*$ :** Las tres versiones producen intervalos de confianza empíricamente consistentes con los coeficientes verdaderos  $\beta^*$ . La implementación automatizada (`bs_auto.py`) logra una cobertura del 93.69 %, mientras que las implementaciones manuales (`bs_sklearn.py` y `bs_numpy.py`) alcanzan un 91.69 %. Aunque la cobertura se encuentra ligeramente por debajo del 95 % teórico, esto es un comportamiento esperado debido al reducido número de remuestreos exigido ( $B = 48$ ) [cite: 1]. Pese a ello, las tres iteraciones capturan exitosamente la inmensa mayoría de los coeficientes, confirmando la validez estructural del algoritmo.

### Equivalencia entre versiones:

- `bs_sklearn.py` y `bs_numpy.py` son estrictamente equivalentes. Al ejecutar la validación, se comprobó que ambas implementaciones son matemáticamente idénticas. Dado que utilizan el mismo arreglo de semillas (`seed_b`) para el generador `np.random.default_rng`, extraen exactamente las mismas muestras en cada iteración  $b$ . En consecuencia, sus estimaciones convergen sin discrepancias.
- `bs_auto.py` es equivalente a nivel estadístico (logrando una cobertura similar del  $\sim 93\%$ ), pero sus intervalos exactos difieren. Esto ocurre porque `BaggingRegressor` administra su propio estado aleatorio interno para el remuestreo de manera opaca, generando secuencias de índices distintas a las provistas en las versiones manuales.

**Condiciones de reproducibilidad:** Para que los resultados sean estrictamente reproducibles entre distintas ejecuciones, se deben cumplir tres condiciones fundamentales:

1. Fijar la semilla del generador inicial utilizado para la creación sintética de la matriz  $X$  y el vector  $y$ [cite: 1].
2. Instanciar generadores aleatorios locales y aislados (`default_rng`) dentro de cada proceso trabajador de `joblib` para anular cualquier condición de carrera en la memoria compartida.
3. Garantizar una asignación determinista de semillas. El mapeo entre la iteración bootstrap  $b$  y su semilla debe ser invariante (por ejemplo, mediante una lista secuencial estática), asegurando que el remuestreo sea independiente del número de procesos  $p$  utilizados o del orden de asignación del sistema operativo.

Para verificar la correctitud y reproducibilidad de las implementaciones, se puede ejecutar el siguiente comando en la terminal:

```
python src/correctness.py
```

## (d) Backend multiprocessing de joblib

**Creación de procesos y asignación de memoria:** En entornos Linux, como el habilitado a través de WSL, el backend de `joblib` utiliza llamadas al sistema operativo (típicamente variantes de `fork` mediante su gestor interno `loky`) para generar procesos hijos a partir del script principal. Al crear estos nuevos procesos, el sistema operativo no realiza una copia inmediata y exhaustiva de la memoria RAM del proceso padre. En su lugar, emplea una técnica de gestión de memoria denominada *Copy-on-Write* (CoW). Bajo este esquema, los procesos hijos recién creados reciben direcciones virtuales que apuntan a las mismas páginas de memoria física del proceso original. El sistema operativo solo asigna memoria nueva y duplica la información si un proceso hijo intenta sobrescribir o modificar los datos.

**Comportamiento de los arreglos  $X$  e  $y$ :** Al lanzar  $p$  procesos, los arreglos sintéticos  $X$  e  $y$  no se replican  $p$  veces en la RAM del computador[cite: 1]. Dado que la lógica del bootstrapping implementada únicamente requiere leer secciones específicas de los arreglos mediante indexación (`X[indices]`), las estructuras originales permanecen intactas y tratadas como datos de solo lectura. Gracias al mecanismo *Copy-on-Write*, los  $p$  trabajadores paralelos acceden simultáneamente a la única copia física de  $X$  e  $y$  que reside en la memoria del proceso principal. Esto previene un desbordamiento masivo de la memoria RAM (impidiendo que  $X \in \mathbb{R}^{10000 \times 301}$  se copie innecesariamente[cite: 1]) y minimiza el *overhead* temporal al instanciar las tareas en paralelo.

## (e) Actividad del sistema y oversubscription

Los experimentos de este inciso se realizaron en un MacBook Air con macOS (arquitectura ARM64) y 8 cores lógicos. Se utilizó NumPy 2.0.2, joblib 1.5.3 y threadpoolctl 3.6.0. La configuración de NumPy indica que las operaciones BLAS y LAPACK están enlazadas con el framework Apple Accelerate.

Ejecutamos `bs_numpy.py` con  $p \in \{1, 2, 4, 8\}$ . Al observar una ejecución prolongada con  $p = 8$ , el sistema mostró el proceso principal, dos procesos auxiliares de `loky` y ocho procesos trabajadores. Los ocho trabajadores registraron aproximadamente entre 53 % y 66 % de CPU cada uno en la muestra observada. Esto confirma que los remuestreos fueron distribuidos entre procesos distintos, aunque no se alcanzó una utilización sostenida de 800 % de CPU.

Para inspeccionar los threads internos, ejecutamos `threadpool_info()` tanto en el proceso principal como dentro de cada trabajador, después de realizar operaciones de álgebra lineal. En todos los casos el resultado fue una lista vacía. Esto no significa que Accelerate nunca utilice threads internos, sino que en esta configuración `threadpoolctl` no detecta ni reporta el pool de Apple Accelerate. Por esta razón no es posible inferir un valor de  $t$  a partir de esta función, a diferencia de instalaciones de NumPy basadas en OpenBLAS o MKL.

Como diagnóstico adicional, repetimos tres veces cada configuración. Las medianas preliminares sin limitar threads fueron 0.555, 0.555, 0.455 y 0.516 segundos para  $p = 1, 2, 4, 8$ , respectivamente. El estancamiento entre  $p = 4$  y  $p = 8$  muestra que agregar procesos deja de producir beneficios, pero no se observó una degradación pronunciada que permita afirmar que existe *oversubscription*. También probamos `threadpool_limits(limits=1)` y la variable `VECLIB_MAXIMUM_THREADS=1`; ninguna produjo una mejora consistente, lo que concuerda con la falta de evidencia de oversubscription en este equipo.

En general, sí existiría oversubscription si cada uno de los  $p$  procesos lanzara  $t$  threads BLAS y se cumpliera  $pt > p_{\text{máx}}$ . En una instalación compatible con `threadpoolctl` puede corregirse envolviendo el cálculo con `threadpool_limits(limits=1)` o configurando `inner_max_num_threads=1` en el backend `loky`. Para Apple Accelerate puede fijarse `VECLIB_MAXIMUM_THREADS=1` antes de iniciar Python. El script `src/experiment_e.py` deja automatizadas la consulta de los pools, la identificación de los procesos trabajadores y la medición.

## (f) Tiempos de ejecución en función de $p$

El primer computador posee  $p_{\text{máx}} = 8$  cores lógicos. Comparamos las versiones finales `bootstrap_auto_v4`, `bootstrap_sklearn_v3` y `bootstrap_numpy_v2` usando los mismos datos ( $N = 10000$ ,  $k = 300$ ),  $B = 48$  y las mismas semillas. Para cada implementación y cada  $p \in \{1, \dots, 8\}$  se realizaron tres ejecuciones. El orden de las 72 configuraciones fue aleatorizado para reducir el sesgo causado por calentamiento del equipo o por el orden de ejecución. Reportamos la mediana, por ser menos sensible que la media a interferencias transitorias del sistema.

La implementación basada solamente en NumPy fue la más rápida para todos los valores de  $p$ : con un proceso tardó aproximadamente 0.520 s, frente a 8.741 s de `BaggingRegressor` y 9.119 s de la versión manual con `scikit-learn`. La diferencia se debe principalmente a que `LinearRegression` realiza más trabajo general para ajustar cada modelo, mientras que la versión NumPy resuelve directamente las ecuaciones normales.

Las dos implementaciones basadas en `scikit-learn` mejoraron al aumentar  $p$  hasta aproximadamente seis o siete procesos. El mejor tiempo observado fue 2.422 s con  $p = 7$  para `BaggingRegressor` y 2.757 s con  $p = 6$  para `sklearn + joblib`. En cambio, NumPy no

| $p$ | <b>BaggingRegressor</b> | <b>sklearn + joblib</b> | <b>NumPy + joblib</b> |
|-----|-------------------------|-------------------------|-----------------------|
| 1   | 8.741                   | 9.119                   | 0.520                 |
| 2   | 5.711                   | 5.431                   | 0.714                 |
| 3   | 3.838                   | 3.584                   | 0.700                 |
| 4   | 3.673                   | 3.927                   | 0.732                 |
| 5   | 2.825                   | 2.978                   | 0.624                 |
| 6   | 3.002                   | 2.757                   | 0.511                 |
| 7   | 2.422                   | 2.880                   | 0.523                 |
| 8   | 2.937                   | 3.443                   | 0.666                 |

Cuadro 4: Mediana de los tiempos de ejecución  $T(p)$ , en segundos, para tres repeticiones en el primer computador.



Figura 1: Tiempo mediano de ejecución en función de  $p$ . Las barras se extienden entre el mínimo y el máximo de las tres repeticiones.

escaló al aumentar los procesos: sus tiempos fluctuaron entre 0.511 s y 0.732 s. En esta versión cada tarea es tan breve que el costo de crear o coordinar procesos, despachar las 48

tareas y administrar sus resultados es comparable al trabajo útil. Finalmente, las tres curvas empeoraron en  $p = 8$ , lo que indica que utilizar todos los cores lógicos no necesariamente minimiza el tiempo debido al overhead y a la competencia por recursos compartidos.

## (g) Speedup $S(p)$ y Eficiencia $E(p)$

A partir de las medianas de la Tabla ??, calculamos para cada implementación:

$$S(p) = \frac{T(1)}{T(p)}, \quad E(p) = \frac{S(p)}{p}.$$

Elegimos como  $T(1)$  la mediana de las tres ejecuciones de la misma implementación con  $p = 1$ : 8.741 s para **BaggingRegressor**, 9.119 s para **sklearn + joblib** y 0.520 s para **NumPy + joblib**. Esta elección mantiene constantes el algoritmo, los datos, las semillas y la instrumentación, por lo que mide escalabilidad fuerte sin mezclar el resultado con una implementación secuencial diferente. Además, usar la mediana evita que una ejecución excepcionalmente lenta o rápida determine toda la curva.

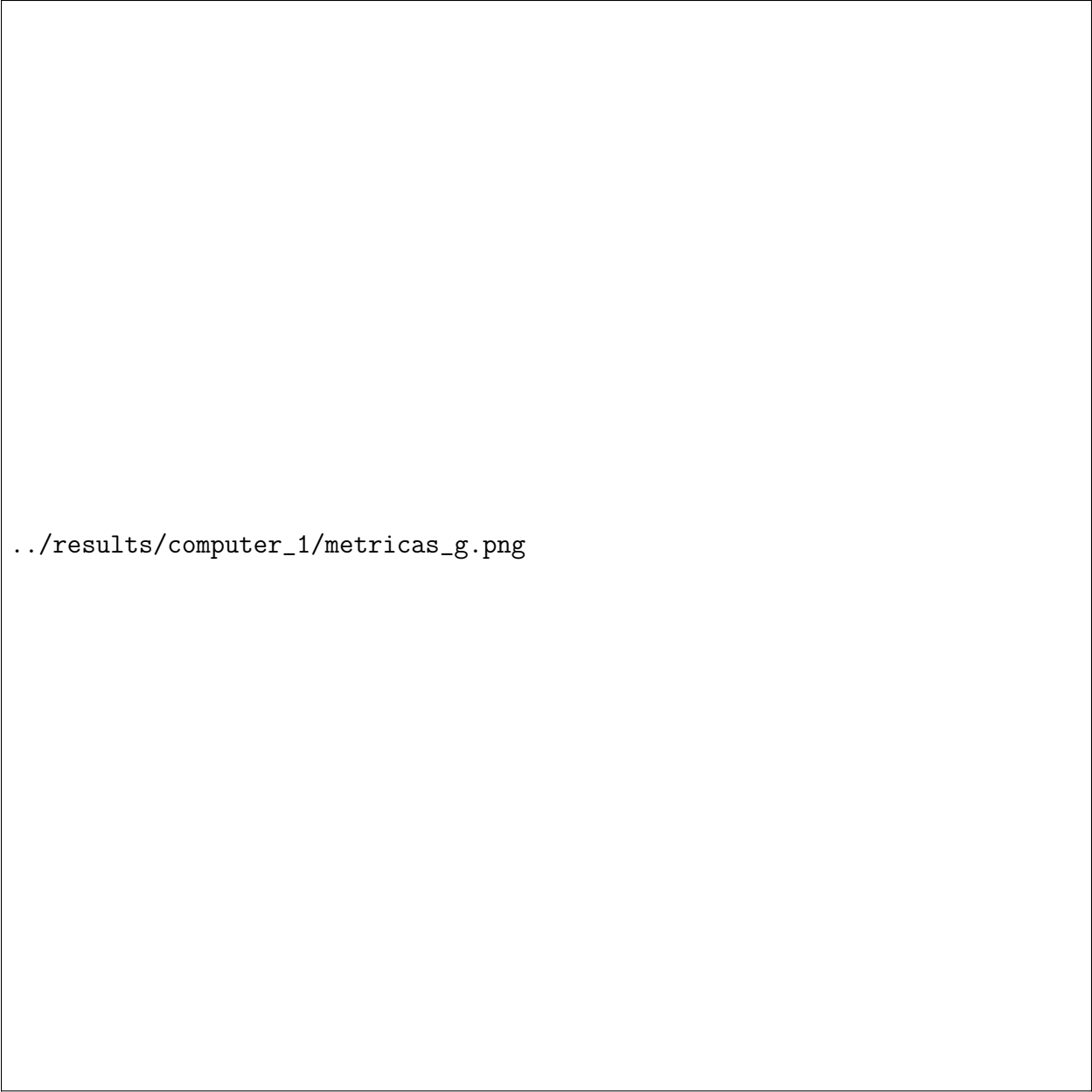
| $p$ | <b>BaggingRegressor</b> |        | <b>sklearn + joblib</b> |        | <b>NumPy + joblib</b> |        |
|-----|-------------------------|--------|-------------------------|--------|-----------------------|--------|
|     | $S(p)$                  | $E(p)$ | $S(p)$                  | $E(p)$ | $S(p)$                | $E(p)$ |
| 1   | 1.000                   | 1.000  | 1.000                   | 1.000  | 1.000                 | 1.000  |
| 2   | 1.530                   | 0.765  | 1.679                   | 0.840  | 0.729                 | 0.365  |
| 3   | 2.278                   | 0.759  | 2.544                   | 0.848  | 0.744                 | 0.248  |
| 4   | 2.380                   | 0.595  | 2.322                   | 0.581  | 0.711                 | 0.178  |
| 5   | 3.094                   | 0.619  | 3.063                   | 0.613  | 0.834                 | 0.167  |
| 6   | 2.912                   | 0.485  | 3.307                   | 0.551  | 1.019                 | 0.170  |
| 7   | 3.609                   | 0.516  | 3.166                   | 0.452  | 0.995                 | 0.142  |
| 8   | 2.976                   | 0.372  | 2.648                   | 0.331  | 0.782                 | 0.098  |

Cuadro 5: Speedup y eficiencia calculados a partir de los tiempos medianos.

Ninguna implementación alcanzó la curva ideal. **BaggingRegressor** obtuvo su mayor speedup en  $p = 7$ , con  $S(7) = 3,609$  y  $E(7) = 0,516$ . La versión manual con scikit-learn alcanzó su máximo en  $p = 6$ , con  $S(6) = 3,307$  y  $E(6) = 0,551$ . En ambas versiones el trabajo de ajustar 48 regresiones es suficiente para amortizar parte del costo de los procesos, pero la eficiencia disminuye al crecer  $p$  debido a creación y coordinación de procesos, distribución de tareas, movimiento de resultados y competencia por memoria.

La versión NumPy presenta un comportamiento distinto. Su tiempo con un solo proceso ya es muy bajo, de modo que el overhead paralelo domina: para casi todos los valores se obtuvo  $S(p) < 1$ . El valor  $S(6) = 1,019$  representa una diferencia de solo 0.010 s respecto de  $T(1)$  y está dentro de la variabilidad observada, por lo que no constituye evidencia de speedup superlineal. Su eficiencia cae desde 0.365 con  $p = 2$  hasta 0.098 con  $p = 8$ . Por lo tanto, en este computador la implementación NumPy es la más rápida en tiempo absoluto, pero su configuración más razonable es  $p = 1$ ; paralelizarla no entrega una mejora robusta para  $B = 48$ ,  $N = 10000$  y  $k = 300$ .





../results/computer\_1/metricas\_g.png

Figura 2: Tiempo, speedup y eficiencia observados. En el panel de tiempo, las líneas discontinuas corresponden a  $T(1)/p$  para cada implementación; en los otros paneles, la curva ideal es  $S(p) = p$  y  $E(p) = 1$ .

### (h) Overhead $T_o(p)$

Una vez obtenidos los datos del inciso g, podemos calcular el overhead  $T_o(p)$  en función de la cantidad de workers, luego posteriormente analizar los 3 métodos. Mientras en el inciso anterior se buscaba ver cuanto se ganaba al aumentar la cantidad  $p$  de procesos, con métricas como el speedup y la eficiencia, en este inciso se buscará cuanto cuesta paralelizar, y demuestra que hay costo y trabajo adicional asociado a la paralelización. La forma de medirlo

es con la siguiente formula

$$T_o(p) = pT_p - T_1$$

Para lo siguiente se creó un código que calculará los overhead por cada método y fuera aumentando los valores de p. Los resultados se pueden observar en las siguientes tablas

| $p$ | <b>BaggingRegressor</b> | <b>sklearn + joblib</b> | <b>NumPy + joblib</b> |
|-----|-------------------------|-------------------------|-----------------------|
| 1   | 0.000                   | 0.000                   | 0.000                 |
| 2   | 2.681                   | 1.743                   | 0.907                 |
| 3   | 2.773                   | 1.633                   | 1.580                 |
| 4   | 5.951                   | 6.588                   | 2.408                 |
| 5   | 5.386                   | 5.769                   | 2.600                 |
| 6   | 9.271                   | 7.424                   | 2.545                 |
| 7   | 8.213                   | 11.042                  | 3.140                 |
| 8   | 14.756                  | 18.428                  | 4.804                 |

Cuadro 6: Overhead  $T_o(p) = pT(p) - T(1)$ , en segundos, calculado a partir de los tiempos medianos de la Tabla ??.



Figura 3: Overhead  $T_o(p)$  en función de  $p$  para las tres implementaciones. La línea discontinua en  $T_o(p) = 0$  corresponde al caso ideal, en que paralelizar no agrega trabajo adicional.

**(i) Variación de procesos y threads internos**

**(j) Comparación entre computadores**

## **Declaración de uso de Inteligencia Artificial**

Se utilizó OpenAI Codex como apoyo para revisar y optimizar el código, diseñar los experimentos y redactar el informe. Los integrantes del grupo revisaron el código y validaron experimentalmente los resultados presentados.